

Cortical-Column Memory: A Thousand-Brains-Inspired Architecture for Structured Agent Memory

System Design, Implementation, and Preregistered Evaluation Protocol

James Booth

Independent researcher

<https://github.com/jmsbooth/cortical-column-memory>

September 2026

v0.1.0 · Architecture and Preregistered Evaluation Protocol · Preprint – not peer reviewed

Abstract

Persistent memory is an increasingly important component of AI-agent systems, yet common approaches often rely on retained transcripts, summaries, similarity retrieval, or flat episodic records. This paper proposes Cortical-Column Memory (CCM), a model-agnostic, agent-internal memory architecture inspired by selected computational principles associated with Thousand Brains Theory. CCM organizes multiple bounded memory/modeling units around structured observations, provenance, temporal history, reference frames, persistent local state, evidence-bearing hypotheses, bounded recall, state reconciliation, and optional active information acquisition. CCM does not model cortical biology and does not claim equivalence between software modules and biological cortical columns. The v0.1.0 release publishes the architecture, reference implementation, tests, deterministic synthetic validation environments, memory-focused baselines, preregistered hypotheses, and machine-readable manifests before the full empirical campaign is completed. Whether this organization improves memory quality, agent reasoning, or resource efficiency remains an empirical question; model-scale and multi-agent composition are secondary research dimensions.

Publication state. The registered experimental campaign has not been completed. This release publishes the architecture, implementation, hypotheses, baselines, metrics, and decision criteria before final data collection. The only executable measurements included at release time are software smoke-test artifacts, clearly labelled as such in the repository.

1 Introduction

AI agents operate across observations, actions, and changing world state. Their current prompt is not persistent memory: a long transcript may contain irrelevant material, a summary may erase the source of a claim, similarity retrieval may mix incompatible times, and a flat episodic record may not expose relations or supersession. These failures can force a reasoning model to repeatedly reconstruct state that an agent could instead maintain explicitly.

This paper asks whether persistent agent memory should be organized primarily as multiple bounded structured models of an observed world rather than as retained text or retrieval over historical text. The proposed answer is Cortical-Column Memory (CCM), a model-agnostic memory architecture in which local modeling units maintain observations relative to reference frames, preserve temporal history and provenance, derive state and hypotheses, and reconcile partial representations into bounded recall context.

1.1 The architectural question

The central question is:

Does structured modular world-state memory improve relevant recall, temporal consistency, relational reconstruction, provenance, and downstream agent performance compared with conventional memory architectures under matched task and resource conditions?

The Thousand Brains literature motivates the design vocabulary of local models, reference-frame-relative structure, partial representations, and active interaction [5, 6]. CCM makes an engineering interpretation of selected principles. It does not claim that a software column is a cortical column, that the implementation reproduces neuroscience, or that TBT is necessary for the architecture to be useful.

1.2 Contributions and boundaries

The contributions of this release are:

1. a memory-first CCM architecture in which an agent may contain multiple bounded columns;
2. explicit contracts for observations, relations, state, hypotheses, transitions, predictions, reference frames, temporal validity, recall, context assembly, and reconciliation;
3. a dependency-free reference implementation with a deterministic symbolic adapter and a provider-neutral model adapter interface;
4. a preregistered evaluation matrix centered on conventional memory baselines, memory-specific tasks, memory-quality metrics, resource costs, and component ablations; and
5. a reproducible validation harness that emits provenance-bearing traces and machine-readable manifests without fabricated campaign results.

Small-model efficiency and multi-agent composition remain useful secondary questions. A multi-agent system may place one independent CCM instance inside each agent, but CCM itself does not define that topology. Similarly, a stronger or frontier model is another model class for measuring interaction effects, not the target that CCM is expected to catch.

1.3 Research questions

The preregistered questions are:

RQ1 – memory quality. Does CCM improve retrieval and reconstruction of relevant historical state?

RQ2 – state consistency. Does CCM improve temporal, relational, and provenance consistency over long interactions?

RQ3 – reference frames. Do explicit reference frames add value beyond structured memory without frame constraints?

RQ4 – modular models. Do bounded local models reduce irrelevant-memory contamination relative to a flat store?

RQ5 – active acquisition. Does active evidence acquisition resolve incomplete or contradictory state?

RQ6 – resources. What storage, latency, context, compute, and coordination overhead does CCM introduce?

RQ7 – model scale. Does the relative benefit vary across model capability classes?

RQ8 – composition. Exploratorily, can independent CCM-equipped agents exchange bounded memory-derived state without shared global memory?

The release is therefore a technical research artifact and preregistered protocol. The full campaign is pending, and all checked-in smoke measurements are software validation rather than evidence for these hypotheses.

2 Background and related terminology

2.1 Thousand Brains Theory

Hawkins, Ahmad, and Cui describe a theory in which cortical columns learn models of objects and their structure relative to reference frames [5]. Hawkins et al. later develop a broader framework involving grid-cell-like location representations and object-centric reference frames [6]. These are theoretical and hypothesis-generating contributions, not specifications for language-agent memory.

2.2 Thousand Brains Project and Monty

The Thousand Brains Project and its Monty implementation are a separate, evolving sensorimotor learning effort [2, 9]. CCM does not reimplement Monty and does not claim that TBT, TBP/Monty, HTM, SDRs, and CCM are interchangeable. The relation is one of architectural inspiration and explicit engineering separation.

2.3 HTM and sparse distributed representations

Hierarchical Temporal Memory is an algorithmic lineage associated with sparse distributed representations, sequence memory, and cortical-learning algorithms [1, 4]. A dense or sparse vector embedding is not automatically an SDR. The v0.1.0 implementation uses structured symbolic relations and no SDR implementation claim.

2.4 LLM memory architectures

LLM-agent memory systems expose several useful design points. MemGPT treats memory and context as an operating-system-like resource boundary [11]; Generative Agents combine observations, reflection, and retrieval in an interactive simulation [12]. These systems motivate persistent memory and context management, while CCM focuses on immutable provenance-bearing observations, temporal validity, bounded local models, and explicit reconciliation.

2.5 Retrieval, episodic memory, and knowledge graphs

Retrieval-augmented generation demonstrates how external retrieval can support knowledge-intensive generation [10]. Vector retrieval is therefore a primary comparison point, but similarity alone does not specify temporal validity, current-state derivation, or contradiction handling. Knowledge graphs provide typed graph data models, query languages, and validation concepts [8]; CCM is narrower, treating an ontology as optional and storing observed instances over time rather than defining an ontology itself. Episodic and semantic memory remain useful system-level categories, but the protocol compares their observable contracts rather than assuming a biological mapping.

2.6 World models and active information

World-model work shows how learned systems can maintain compressed spatial and temporal representations of an environment [7]. CCM uses “world model” cautiously: its implementation provides state-transition and prediction contracts, but the v0.1.0 symbolic harness does not establish a learned world model. Active evidence is treated as an information-acquisition policy that selects observations when memory state is incomplete or contradictory.

2.7 Multi-agent inference and test-time computation

Self-consistency, multi-agent debate, and mixture-of-agents systems study how multiple inference traces can improve answers [3, 14, 15]. Test-time compute scaling is another secondary comparison dimension [13]. CCM retains these as exploratory composition or inference baselines. They are not the primary benchmark because the central object of this paper is persistent agent memory, not answer aggregation.

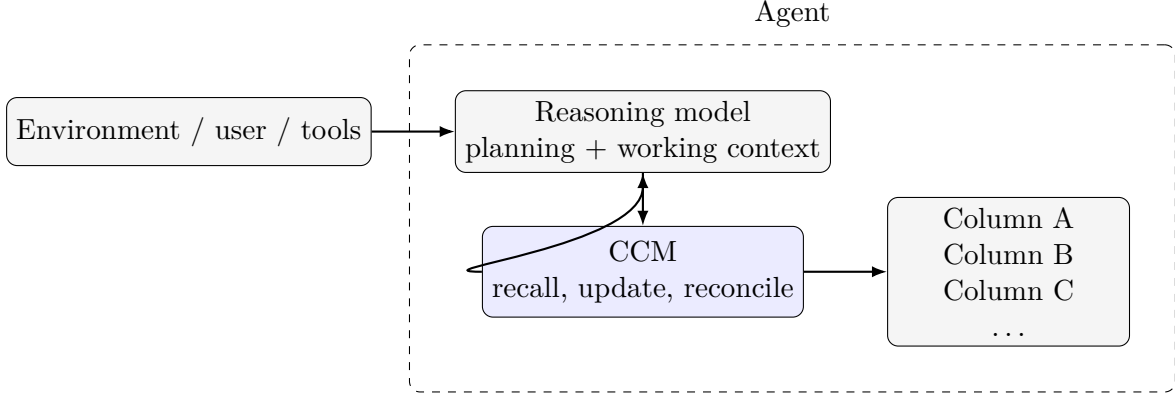


Figure 1: Canonical CCM relationship. Columns are bounded memory/modeling units inside an agent, not autonomous agents.

2.8 Engineering boundary

The mapping from these literatures to CCM is intentionally one-way. The cited work motivates modular state, retrieval, history, reference frames, and active interaction. CCM implements those ideas as data structures, APIs, and tests. It does not assert that the interfaces reproduce neurons, layers, grid cells, or biological learning, and its claims must ultimately be judged by its own engineering evidence.

3 CCM architecture

3.1 Agent-internal architecture

The canonical relationship is shown in Figure 1. A CCM column is not an agent. An agent contains its reasoning model, instructions, planning, tools, working context, and optionally one CCM instance composed of multiple bounded columns. CCM returns bounded memory context to the reasoning model; it does not replace the agent’s reasoning loop.

The default conceptual mode is shared-model: all columns may call the same parent-agent model while maintaining distinct local state. A provider-neutral adapter also permits heterogeneous models. The architecture is independently usable by a single agent and does not require a multi-agent orchestrator or TPAA.

3.2 Multi-agent composition

Multi-agent composition is a separate architectural extension, not the identity of CCM. Figure 2 shows one possible topology in which each agent owns an independent memory instance and exchanges only bounded, provenance-bearing state through an outer coordination layer.

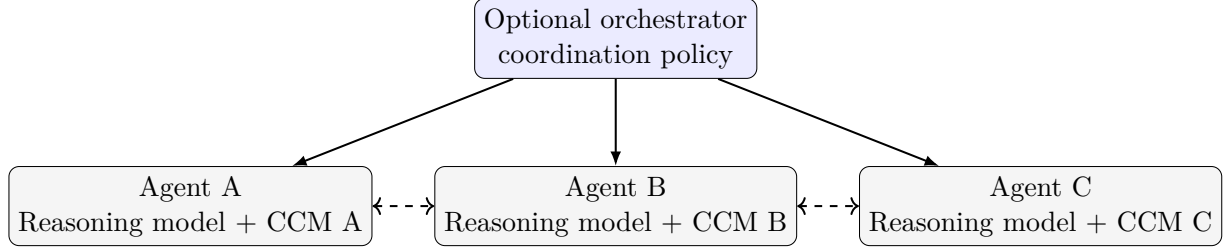


Figure 2: Secondary multi-agent composition.

CCM does not require this topology or shared global memory.

3.3 Reference frames

A CCM reference frame answers: relative to what structured context does an observation have meaning? Formally,

$$F = (f, k, o, s, T, O),$$

where f is a stable identity, k is a frame class, o is the origin, s is the current state, T is a set of validated transitions, and O maps locations to observation identifiers. A transition $(a, s_i, s_j) \in T$ is valid only when the current state is s_i ; an observation is attachable only when its frame and location are valid.

Frames may be object-, actor-, process-, temporal-, spatial-, relational-, workflow-, or ontology-relative. They are not necessarily spatial and are not agent personas. A frame can organize a mug’s object-local handle location, an incident’s authorization workflow, or a temporal assignment history.

3.4 Columns

A column is a bounded persistent modeling unit responsible for one coherent subset or perspective of world state. It may accept observations, place them in a frame, maintain local state and relations, retain temporal history, track conflicting observations, derive hypotheses, identify missing information, and emit a bounded external representation. It need not independently operate as a full autonomous agent.

The inspectable interface is:

```

observe(observation)
update_hypotheses(candidates)
propose_action(available_actions)
emit_state()
receive_peer_state(peer_state)
vote()           # optional reconciliation algorithm
commit(status)

```

The interface separates local update from communication. Peer state is serializable and bounded; hidden reasoning is not exchanged. A column can contribute to committed, abstained, unresolved,

or complementary state.

3.5 Memory model

CCM distinguishes immutable historical observations from current interpreted state. The principal object classes are Observation, Relation, State, Hypothesis, Transition, and Prediction. An observation contains stable identity, content, frame and location, source, provenance, sequence, confidence, validation state, observed time, recorded time, validity interval, optional supersession, and relations. A Relation is a typed subject–predicate–object assertion with its own provenance and validity. State is derived and replaceable; history is not.

For example, an incident frame can contain the actor, session, device, credential, resource, action, time, and authorization context. The ontology, if supplied, defines valid entity and predicate types. CCM stores instances: “Session S19 accessed Resource R4 at 14:32 using Token T7.” A provenance-bearing recall can return the relevant state and source observation without exposing unrelated history.

3.6 Recall and context assembly

The agent-facing interface is:

```
recall(query, current_context, budget)
```

The result contains relevant observations, current state, supporting IDs, temporal context, provenance, unresolved conflicts, hypotheses, confidence, and an auditable token count. Context assembly converts that result into bounded model input. The experimental harness measures recall precision, recall, provenance coverage, relevant-context density, context tokens, candidate count, retrieval latency, and update cost. CCM does not assume that raw memory is copied into the prompt.

3.7 Reconciliation rather than voting

Local states may be compatible, complementary, contradictory, unresolved, or unknown. The reconciliation operation classifies these outcomes without forcing a winner. Structured quorum voting is retained as one optional algorithm for a downstream decision. It is not required for every recall and is not the defining mechanism of CCM.

3.8 Active evidence

When recalled state is incomplete or contradictory, a column may identify a missing observation and an optional policy may select an available action by expected information gain per cost. The returned observation is validated, appended with source and provenance, and included in the next recall. An action cannot create evidence without an environment response.

Table 1: CCM outcomes.

Outcome	Meaning
Agreement	Local states are compatible.
Complementary	Local states contribute non-overlapping information.
Contradiction	Same frame/location contains incompatible claims.
Unresolved	Evidence is insufficient to choose or reconcile.
Unknown	No derived state is available.
Committed	An optional decision rule accepts a state.

3.9 Primary algorithm

Algorithm 1 is implementation-independent pseudocode. It makes recall, reconciliation, temporal history, and disagreement explicit.

1. Initialize an agent’s bounded columns, reference frames, append-only memory, and run manifest.
2. Accept externally returned observations with source, provenance, sequence, and temporal metadata.
3. Recall a bounded context for the agent query; assemble only the required context for the reasoning model.
4. Each column updates local state and hypotheses, then emits a bounded structured state.
5. Reconcile emitted states as agreement, complementary, contradiction, unresolved, or unknown.
6. Optionally apply a structured quorum or another decision rule; never convert a hypothesis into an observation.
7. If uncertainty remains and budget permits, select and execute one evidence action, append its result, and repeat.
8. Otherwise return the bounded state with provenance and an explicit outcome.

3.10 Invariants

The implementation and tests enforce: observations are immutable and provenance-bearing; hypotheses cannot masquerade as observations; current state never deletes history; temporal validity and supersession are explicit; columns do not silently mutate one another; contradictory and unknown outcomes remain visible; recall output is bounded and provenance-linked; and manifests record configuration, model/task versions, seed, hardware, resources, and Git commit.

4 Problem definition and memory operations

4.1 Agent memory problem

An agent operates over time, but its current context is not equivalent to persistent memory. Long interaction histories introduce irrelevant context, retrieval ambiguity, temporal inconsistency, repeated reconstruction, provenance loss, and context-window pressure. CCM treats memory as an explicit state-maintenance problem rather than assuming that a reasoning model can reconstruct all relevant world state from text.

4.2 Conventional memory tradeoffs

Full transcripts retain detail but consume context. Summaries reduce cost but may erase provenance and contradictions. Vector retrieval improves selectivity but does not, by itself, represent temporal validity or typed relations. Episodic and semantic stores offer stronger structure but vary in their treatment of supersession and unresolved evidence. Knowledge graphs encode valid relationships but are not automatically a record of observed instances. CCM is evaluated against these alternatives rather than dismissing them.

4.3 Formal memory objects

The conceptual schema distinguishes six object classes:

Observation. An externally acquired fact or state with identity, source, provenance, reference frame, location, sequence, confidence, validation state, observed time, recorded time, and optional validity interval. Observations are immutable.

Relation. A typed subject–predicate–object relationship with provenance, validity, and confidence.

State. The current interpreted state within a frame, derived from one or more observations. Updating state does not delete its supporting history.

Hypothesis. An unconfirmed candidate explanation or state. Model-generated hypotheses cannot be promoted to external observations.

Transition. A validated or hypothesized state change with explicit source, action, and target.

Prediction. An expected future observation under a hypothesis, retained separately from observed evidence.

4.4 Historical and current state

Historical observations and current model state are not equivalent. If Device A is owned by User X at T_1 and reassigned to User Y at T_2 , CCM retains both observations while the current interpretation resolves to User Y. The implementation represents this distinction with validity intervals and

supersession metadata; a late-arriving observation is appended rather than overwriting the event history.

4.5 Recall and context assembly

The primary agent-facing operation is:

```
recall(query, current_context, budget)
```

It returns a bounded memory context containing relevant current state, supporting observations, temporal context, provenance, unresolved conflicts, relevant hypotheses, confidence, and optionally recommended additional evidence. Context assembly then converts that structured result into a bounded representation for the reasoning model. The protocol measures recalled evidence coverage, provenance coverage, irrelevant-context ratio, context tokens, recall latency, and update cost.

4.6 Reconciliation

Recall does not require voting. States emitted by local models may be compatible, complementary, contradictory, unresolved, or unknown. The implementation exposes a reconciliation classifier for these outcomes. A quorum-based structured vote remains an optional downstream commit algorithm under evaluation; CCM is not defined by quorum voting.

4.7 Research boundaries

CCM is independent of TPAA and any particular agent framework, model provider, transport, or deployment topology. It can be embedded inside a single agent or instantiated independently across several agents. Thousand Brains Theory supplies architectural inspiration, not a biological implementation specification. The engineering contribution stands or falls on the memory contracts and empirical evidence reported by this protocol.

5 Preregistered evaluation protocol

The authoritative registration is in `preregistration/`. The full configuration is the v0.1.0 file under `registered-configs/`; the smoke configuration is separate and is not a substitute for the campaign. The primary comparison is memory quality under matched task, history, model, and resource conditions.

5.1 Primary memory baselines

ID	Definition
M0	No persistent memory; current task context only.

M1	Full-history memory.
M2	Periodically summarized textual memory.
M3	Vector-similarity retrieval over historical observations.
M4	Structured key/value or episodic memory.
M5	Knowledge-graph memory where practical.
M6	Full CCM: bounded local models, frames, history, recall, reconciliation, and optional active evidence.
M7	CCM plus an optional ontology substrate; exploratory if implemented.

The deterministic v0.1.0 adapter implements transparent symbolic controls for M0–M6. It does not claim to be a learned summarizer, embedding model, or production knowledge graph. The full registered campaign should use explicit model/runtime implementations where resources permit.

5.2 CCM ablations

Ablation	Removed or changed mechanism
CCM-NoRF	Explicit reference-frame constraints.
CCM-NoModules	Multiple bounded columns; one local module remains.
CCM-NoTemporal	Temporal recall and current-state constraints.
CCM-NoReconciliation	Reconciliation-driven structured commit.
CCM-NoActive	Active evidence acquisition.
CCM-FlatLedger	Frame-aware organization; flat structured-ledger comparison.

5.3 Memory task classes

The deterministic suite prioritizes persistent factual recall, temporal state, relational recall, conflicting evidence, state replacement/supersession, provenance recall, cross-frame integration, distractor resistance, long-horizon continuity, active evidence, incomplete information, and pure reasoning control. Each task has a stable ID, hidden answer, candidate hypotheses, explicit observations or actions, and required evidence where applicable.

5.4 Metrics

Primary metrics are recall precision, recall recall, state reconstruction accuracy, temporal accuracy, relation accuracy, provenance accuracy, contradiction preservation, false-memory rate, stale-state rate, and relevant-context density. Secondary metrics include downstream task success, unsupported-memory rate, context tokens, retrieval/update latency, storage bytes, relation count, index size, candidates examined, communication, calibration, abstention, unresolved outcomes, and failure taxonomy. Smoke tables expose these fields but do not constitute campaign results.

5.5 Model scale and composition

Model scale is a secondary dimension. Where practical, the same memory architecture should be evaluated across small local, larger open, and hosted/frontier model classes, measuring relative improvement from each model’s baseline memory condition rather than asking whether a small model

catches a frontier model. Multi-agent composition is exploratory: independent CCM-equipped agents may exchange bounded memory-derived state without requiring a shared global store.

5.6 Resource accounting

The harness records model invocations, input/output tokens, wall-clock latency, communication, bytes stored, object and relation counts, index size, recall candidates examined, context tokens produced, retrieval latency, update cost, and unavailable hardware/energy/API fields. Pareto analysis therefore compares memory quality against storage, retrieval latency, context, and compute.

5.7 Statistical decision rules

The unit of analysis is a task instance, paired by task ID and seed. The registered configuration uses three independent seeds and ten instances per task class. The analysis reports raw differences, 95% percentile bootstrap confidence intervals using 10,000 resamples, paired effect sizes, and Holm correction across confirmatory comparisons. A minimum practically meaningful difference is 0.05 on the relevant bounded metric. Failures and timeouts remain in denominators; there is no silent outlier removal or outcome-dependent stopping.

5.8 Negative results and provenance

All registered instances, unresolved states, contradictions, abstentions, timeouts, and unavailable resource measurements remain in the analysis. Run manifests record configuration hash, Git commit, model/task versions, seed, hardware, timestamp, and raw output. Private chain-of-thought is neither required nor stored. The protocol makes no claim until the full campaign replaces the smoke status with traceable results.

6 Reference implementation

The repository is a standard-library-only Python reference implementation. The conceptual boundaries are explicit even where the v0.1.0 directory layout remains compact:

ccm/schemas. Stable contracts for observations, relations, state, hypotheses, transitions, predictions, votes, actions, and column state.

ccm/memory. Append-only observation history, temporal queries, supersession, and provenance-bearing write/retrieve operations.

ccm/recall. Bounded lexical recall, current-state selection, conflict visibility, and context assembly.

ccm/reference_frames. Valid state transitions and frame/location constraints.

ccm/column. Bounded local modeling-unit interface. A column is not an agent.

ccm/consensus. Reconciliation outcomes and optional structured quorum voting.

ccm/active_evidence. Information-gain-per-cost action selection.

ccm/adapters. Provider-neutral model adapter contract, including an injected JSON-boundary LLM adapter with no network dependency.

ccm/orchestration. Local execution loop that records recall, context assembly, reconciliation, optional action, and explicit termination.

experiments and evaluation. Deterministic task worlds, M0–M6 controls, CCM ablations, manifests, memory metrics, resource accounting, and statistical utilities.

The symbolic `ToyModel` is intentionally deterministic and inspectable. It supports unit, invariant, integration, and smoke validation; it is not described as a language-model result. An actual model can implement the injected adapter interface while preserving the observation/provenance boundary.

6.1 Testing surface

Tests cover immutable observations, duplicate handling, required provenance, hypothesis/observation separation, historical and current queries, supersession, late temporal validity, valid and invalid reference-frame transitions, and column isolation. They also cover agreement, complementary evidence, contradiction, unknown and unresolved outcomes, bounded recall, distractor rejection, multi-hop state, provenance return, bounded context assembly, deterministic task generation, manifests, and generated smoke record shape.

6.2 Reproducibility

Every run emits JSONL records and a manifest with configuration hash, protocol version, model and task version, seed, hardware metadata, Git commit, timestamps, resource fields, structured trace, and output path. The repository contains no hosted credentials and the smoke path has no network dependency.

7 Results status

The full registered campaign has not been executed. This release therefore does not report a scientific estimate for H1–H10 and does not claim that CCM improves memory quality. The checked-in smoke artifacts are deterministic software-validation outputs used to verify task generation, schema contracts, trace completeness, resource fields, analysis, and paper-table generation.

The generated table in Table 4 is intentionally labelled smoke validation. It exposes memory-oriented fields such as recall, temporal accuracy, provenance, and context tokens; it must not be interpreted as a comparison of language-model capability. A future registered run will replace protocol-state language only after all configured seeds, task instances, baselines, ablations, matched resource views, confidence intervals, effect sizes, and failure analyses are present.

Table 4: Deterministic smoke-validation schema. Values are generated from the checked-in symbolic adapter and are not scientific campaign results.

Strategy	Recall	Temporal	Provenance	Context tokens
CCM-FlatLedger	1.000	0.000	0.833	35.5
CCM-NoActive	0.818	0.000	0.667	31.4
CCM-NoModules	1.000	0.000	0.833	37.9
CCM-NoRF	1.000	0.000	0.833	35.5
CCM-NoReconciliation	1.000	1.000	0.833	35.5
CCM-NoTemporal	1.000	0.000	0.000	0.0
M0	0.000	1.000	0.000	0.0
M1	0.818	1.000	0.000	8.8
M2	0.545	1.000	0.000	4.7
M3	0.818	1.000	0.000	8.8
M4	0.818	1.000	0.000	8.8
M5	0.818	1.000	0.000	8.8
M6	1.000	0.000	0.833	35.5

8 Discussion

Some failures attributed to agent reasoning may arise because the model is repeatedly asked to re-construct world state from incomplete, noisy, semantically retrieved, or compressed history. CCM proposes a different systems boundary: preserve historical observations and provenance, derive current state explicitly, organize local models relative to frames, and return bounded context through a recall interface.

The design has several intended advantages. Temporal semantics separate what was true from what is true now. Provenance makes a recalled claim auditable. Modular local models may reduce irrelevant-memory contamination and permit complementary state rather than a single collapsed summary. Reconciliation can preserve contradictions and unknown outcomes. Active evidence can make a missing observation an explicit next step rather than an unsupported completion.

These advantages have costs. Structured state requires schemas, update policies, conflict handling, indexing, and additional storage. Multiple columns can correlate errors when they share one model. Recall and context assembly introduce latency and implementation complexity. A structured representation can also encode the wrong ontology or miss information that a free-form transcript retained. The protocol therefore measures memory quality jointly with storage, latency, context, and compute.

The model-scale interaction is deliberately secondary. A stronger model may benefit from the same structured memory, and a weaker model may benefit more or less depending on its ability to use the context. The relevant comparison is relative improvement from each model’s matched memory baseline, not a promise that small models catch frontier models. Multi-agent composition is similarly a follow-on question: independent CCM instances may exchange bounded state, but CCM does not define the outer topology.

Ontology integration is optional. An ontology can constrain valid entities and predicates, while CCM retains observed instances and their time-dependent state. A future ontology-backed experi-

ment should be treated as a distinct extension rather than folded into the core definition. Likewise, the “world model” label is limited here to explicit transition and prediction contracts; the symbolic harness does not establish a learned world model.

Finally, the Thousand Brains analogy is a design inspiration with a strict boundary. If TBT is incomplete or revised, CCM should still stand or fall on its own memory contracts, tests, and empirical evidence.

9 Related work and threats to validity

9.1 Novelty matrix

The matrix is a conservative design comparison, not a claim that every system in a category has identical capabilities. “Varies” indicates that the property depends on the implementation.

Approach	Hist.	Rel.	Time	Frame	Prov.	Local	Conflict	Active
Transcript	yes	var.	impl.	no	text	no	var.	no
Summary	comp.	var.	lossy	no	lossy	no	lossy	no
Vector memory	yes	no	var.	no	var.	no	var.	no
KG memory	yes	yes	var.	no	var.	no	var.	no
Episodic memory	yes	var.	var.	var.	var.	var.	var.	var.
TBP/Monty	yes	yes	var.	loc.	var.	mod.	var.	yes
CCM	yes	yes	exp.	exp.	req.	bound.	kept	opt.

CCM’s distinguishing engineering claim is the combination of immutable provenance-bearing observations, explicit temporal/current-state separation, reference-frame constraints, bounded local models, recall/context assembly, and reconciliation that preserves contradiction. The paper does not claim that CCM invented persistent memory, graphs, reference frames, active information acquisition, or multiple agents.

9.2 Threats to validity

Biological analogy. TBT, TBP/Monty, HTM, SDRs, and CCM are distinct. The paper makes no neural or cortical equivalence claim.

Toy adapter. The symbolic adapter validates software contracts, not natural-language capability. Full claims require model-backed runs with exact model/runtime provenance.

Task validity. Synthetic tasks may not represent real agent workloads. The protocol therefore labels them as controlled validation and requires future domain tasks before broad generalization.

Metric validity. Recall precision, temporal accuracy, provenance, and context density are operational measures, not complete measures of useful memory. Downstream task success remains secondary but necessary.

Correlated columns. Shared-model columns are not independent evidence. The manifest records model identity and the protocol includes a no-modules condition.

Retriever bias. Lexical smoke recall is not a vector retriever. M3 must use a real embedding/retrieval implementation in a registered empirical run.

Resource measurement. Local runs may not expose energy, accelerator time, or API cost. Those fields remain unavailable rather than imputed.

Negative outcomes. Contradictions, abstentions, unresolved results, failures, and missing resources remain in denominators. No outcome-dependent stopping or silent removal is permitted.

9.3 Scope and reproducibility

All public claims must be classified as primary literature fact, project implementation fact, interpretation, CCM design choice, or hypothesis. The source bibliography uses primary TBT papers, project sources, memory/retrieval work, and carefully bounded secondary comparisons. The checked-in code, protocol, manifests, tests, and paper build provide the reproducibility surface.

10 Conclusion

CCM is proposed as a modular persistent-memory architecture for AI agents. Its central design proposition is that agent history can be maintained as multiple structured, reference-frame-relative models rather than reconstructed primarily through retained text or similarity retrieval. The architecture separates observations from derived state, preserves temporal history and provenance, supports bounded recall and context assembly, and reconciles partial models without requiring every recall to be a vote.

The v0.1.0 release defines the architecture, reference implementation, invariants, deterministic validation tasks, memory-focused baselines, and preregistered protocol. Whether this organization improves memory quality, agent reasoning, or resource efficiency remains an empirical question. Small-model interaction and multi-agent composition are secondary extensions, and TPAA is neither required by nor part of the CCM definition.

A Preregistration summary

This appendix summarizes the revised protocol. The YAML and JSON files in `preregistration/` are authoritative for execution.

A.1 Registered hypotheses

ID	Registered statement
H1	CCM improves relevant-memory recall over conventional memory baselines under matched conditions.
H2	CCM reduces stale or temporally incorrect state retrieval.

H3	Explicit reference frames improve relational/state reconstruction over CCM without frame constraints.
H4	Modular local models reduce irrelevant-memory contamination relative to a flat structured store.
H5	CCM preserves contradictory and unresolved evidence better than single-summary memory.
H6	CCM improves provenance accuracy over conventional memory baselines.
H7	CCM reduces context tokens for equivalent or improved task success.
H8	CCM’s relative benefit varies by base-model capability.
H9	Exploratory: active evidence improves incomplete-memory resolution.
H10	Exploratory: independent CCM-equipped agents exchange bounded memory-derived state without shared global memory.

A.2 Analysis commitments

Three independent seeds and ten instances per task class are registered. Primary metrics are recall precision/recall, state reconstruction, temporal accuracy, relation accuracy, provenance accuracy, contradiction preservation, false-memory rate, stale-state rate, and relevant-context density. Report raw effects, 95% bootstrap confidence intervals, paired effect sizes, and practical significance; apply Holm correction across confirmatory tests; retain timeouts and failures; perform no silent outlier removal or outcome-dependent stopping.

A.3 Negative-results and version policy

Registered results will be published whether hypotheses are supported, qualified, or rejected. A component that contributes nothing or increases resource cost will be reported. Corrections that do not alter registered elements may be v0.1.x releases with changelog entries. Changes to hypotheses, metrics, baselines, exclusions, or thresholds require a deviation record and a new protocol version. A completed empirical release requires all registered seeds, raw and processed data, generated tables and figures, matched-budget analyses, ablations, failure taxonomy, resource-quality Pareto analysis, and evidence-based abstract and conclusion updates.

References

- [1] Subutai Ahmad and Jeff Hawkins. How do neurons operate on sparse distributed representations? A mathematical theory of sparsity, neurons, and semantic memory. *arXiv preprint arXiv:1601.00720*, 2016. <https://arxiv.org/abs/1601.00720>.
- [2] Viviane Clay, Niels Leadholm, and Jeff Hawkins. The Thousand Brains Project: A new paradigm for sensorimotor intelligence. *arXiv preprint arXiv:2412.18354*, 2024. <https://arxiv.org/abs/2412.18354>.
- [3] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023. <https://arxiv.org/abs/2305.14325>.
- [4] Jeff Hawkins and Subutai Ahmad. Hierarchical temporal memory including HTM cortical learning algorithms. Numenta technical report, 2010. <https://www.numenta.com/assets/pdf/whitepapers/hierarchical-temporal-memory-cortical-learning-algorithm-0.2.1-en.pdf>.

- [5] Jeff Hawkins, Subutai Ahmad, and Yuwei Cui. A theory of how columns in the neocortex enable learning the structure of the world. *Frontiers in Neural Circuits*, 11:81, 2017. <https://doi.org/10.3389/fncir.2017.00081>.
- [6] Jeff Hawkins, Marcus Lewis, Mirko Klukas, Scott Purdy, and Subutai Ahmad. A framework for intelligence and cortical function based on grid cells in the neocortex. *Frontiers in Neural Circuits*, 12:121, 2019. <https://doi.org/10.3389/fncir.2018.00121>.
- [7] David Ha and Juergen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. <https://arxiv.org/abs/1803.10122>.
- [8] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021. <https://doi.org/10.1145/3447772>.
- [9] Niels Leadholm, Viviane Clay, Scott Knudstrup, Hojae Lee, and Jeff Hawkins. Thousand-brains systems: Sensorimotor intelligence for rapid, robust learning and inference. *Neural Computation*, 38(6):845–896, 2026. <https://doi.org/10.1162/NECO.a.1508>.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Sebastian Rocktaeschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2005.11401>.
- [11] Charles Packer, Vivian Fang, Shishir Bothmann, Lin Xu, P. S. Ammanamanchi, Juan Gonzalez, Ruslan Salakhutdinov, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. <https://arxiv.org/abs/2310.08560>.
- [12] Joon Sung Park, Joseph O’Brien, Carrie J. Cai, Meredith Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. <https://doi.org/10.1145/3586183.3606763>.
- [13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. <https://arxiv.org/abs/2408.03314>.
- [14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2203.11171>.
- [15] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024. <https://arxiv.org/abs/2406.04692>.